Where AI tokens go: 13 open measurements of LLM and coding-agent token costs

TokenSave · Technical report, version 1 · 19 October 2026 · DOI 10.5281/zenodo.23467575 · https://tokensave.app/data/ ·
License CC BY 4.0

Abstract. We publish thirteen small, fully reproducible measurements of how many tokens large language models and AI coding agents consume, with raw logs and scripts, each archived on Zenodo with a DOI. Six measure tokenizers directly: the same content across 41 natural languages, 8 data formats, 10 image resolutions, 10 programming languages and PDF versus plain text, on Claude, Gemini and GPT tokenizers. Seven measure agents end to end: Claude Code (Opus 5.5) and Codex CLI on the same tasks, varying effort level, web search, search language, instruction-file size, prompt language and harness features. Three patterns recur. First, in agentic coding the fixed harness dominates: a one-line system prompt with two tools cut median input tokens by 90% on the same task, while switching the prompt language moved them by at most 1.2%. Second, token counts and cost diverge because of prompt caching: plan mode tripled input tokens but raised cost by 26%. Third, tokenizer differences are large in chat-sized text (Korean needs 89 Claude tokens against 49 on GPT) but are diluted inside agent runs. This report summarises the datasets, their methods and their limitations.

1. Motivation

Token counts drive both the price and the usage limits of LLM products, but published figures are usually vendor estimates, single anecdotes or numbers without raw data. We wanted measurements that a reader can rerun: the same input, the same task, several runs, the exact tool versions and the raw output of every run. Each dataset below is small and narrow on purpose; together they show where tokens are spent at the level of the tokenizer and at the level of a coding agent.

2. Methods in brief

Tokenizer measurements (datasets 1–6) count tokens for fixed inputs. GPT counts use OpenAI's `o200k_base` and `cl100k_base` encodings or the published image-patch formula; Claude counts are read from Claude Code usage logs (Opus/Sonnet 5.5 and Haiku 4.5); Gemini counts come from the Google AI Studio token counter (Gemini 3.8 Flash). Inputs are included in each dataset so every count can be reproduced.

Agent measurements (datasets 7–13) run Claude Code 2.1.295 with Opus 5.5 and Codex CLI 0.162 with gpt-6-luna on one Windows laptop, effort medium unless stated, each run in its own empty folder with the prompt sent through standard input. Tokens and cost come from the tools' own JSON output (`modelUsage` and `total_cost_usd` for Claude Code, `turn.completed` for Codex); input tokens include cache reads and cache writes. Generated games were checked in headless Chromium. Runs per setting range from 2 (earliest datasets) to 5.

3. Findings

3.1 Tokenizers

#	Dataset	Main result
1	Token cost by language (41 languages, GPT)	The same prompt needs 34 tokens in English and 35–83 in 40 other languages on `o200k_base`: +3% (Simplified Chinese) to +144% (Punjabi); Korean +44%, Japanese +79%.

2	Token cost by language on Claude, Gemini and GPT	Korean needs 89 tokens on Claude 5.5, 49 on GPT and 48 on Gemini. Median extra cost of 40 non-English languages versus English: +41% Gemini, +52% GPT, +88% Claude 5.5.
3	Token cost by data format	A 20-row table costs 1,336 GPT tokens as indented JSON and 429 as CSV (−68%). CSV was cheapest on every tokenizer; TOON came close but did not beat it.
4	Image token cost by resolution	A 12-megapixel photo costs 1,065 input tokens on Gemini, 4,005 on Claude 5.5 and 14,364 on GPT-6 Astra by default. Gemini charges about the same for any image.
5	Token cost by programming language	Two programs cost 809 GPT tokens in Python and 1,536 in C (+90%). Claude 5.5 needed 43–59% more tokens than GPT for the same code.
6	PDF token cost per page	A text page costs 2,347 tokens as a PDF on Claude 5.5 and 728 pasted (+222%); Gemini charges a flat 560 tokens per page.

3.2 Coding agents

#	Dataset	Main result
7	Claude effort levels	The same coding task used 6,299 output tokens and $0.56 at medium effort on Opus 5.5, and 160,033 tokens and $5.25 at max; high cost 34% more than medium.
8	Claude Code vs Codex	Before doing anything, Claude Code sends 35,771 input tokens and Codex 14,678. On the same task Codex used 52% fewer input and 59% fewer output tokens. 30 extra MCP tools barely changed either.
9	Web search in Claude Code and Codex	A one-line fact question with web search cost 150,414 input tokens in Claude Code and 33,296 in Codex.
10	AI web search by language	Asked about AI in Korean, Japanese, Spanish or Ukrainian, the agents ran 205 of 206 searches in English and cited no pages in the question’s language. For local topics in the local language, Claude Code searched in that language 82% of the time and Codex 50%.
11	CLAUDE.md and AGENTS.md size	A 50 KB CLAUDE.md added 15,985 input tokens to every request. Codex read only the first 32 KiB of a 50 KB AGENTS.md and silently ignored an instruction at the end.
12	Prompt language in coding	The same task prompted in Korean, Japanese, Spanish or Ukrainian stayed within 1.2% of the English median input tokens in both tools; the game text and Claude Code’s comments followed the prompt language.
13	Claude Code harness cost	Median input per task: default 78,723; minimal harness 8,214 (−90%); safe mode with 35 unused plugins off 48,063 (−39%); subagent 534,125 (+578%, cost +128%); plan mode +212% tokens but +26% cost.

4. Cross-cutting observations

The harness dominates agentic cost. In dataset 13 a one-line system prompt with two tools reduced median input from 78,723 to 8,214 tokens on the same task, and all games still worked. In dataset 12 the prompt language, which matters a great deal in chat-sized text (datasets 1 and 2), moved input tokens by at most 1.2%. In dataset 8 Claude Code’s startup context was 35,771 tokens before any work. For agentic coding, the tool’s own instructions, tool descriptions and installed extensions are the main lever.

Token counts are not cost. Prompt caching makes repeated context cheap. Plan mode raised input tokens by 212% but cost by 26%; a project skill doubled input tokens (+101%) but raised cost by 2% (dataset 13). In dataset 11, a 50 KB CLAUDE.md raised the cost of the first request by 79% but of the second by 2%. Reports that quote token counts alone overstate the cost of features that re-read cached context, and understate the cost of features that create new uncached context, such as subagents.

Unused extensions are not free. With 35 plugins installed (376 skills and 19 agents listed to the model) and none used, the same task needed 30,660 more input tokens than in safe mode. Datasets 8 to 12 were measured

on the same machine with the same plugins, so their absolute Claude Code figures include this overhead; comparisons inside each dataset are unaffected.

Language effects depend on the layer. Tokenizers differ widely by language (Kannada +577% on Claude 5.5 against +71% on Gemini in dataset 2), and agents search in English for global topics whatever the question language (dataset 10). Inside an agentic coding task the language of a short prompt hardly matters (dataset 12).

5. Limitations

- One Windows laptop, one account per tool, and tool versions current in October 2026; results will change with new models and releases.
- Few runs per setting (2 to 5) and mostly one small task; larger projects may favour subagents, plans or skills that this task never needed.
- Claude and Gemini counts are read from product logs and counters rather than open tokenizers; GPT image counts use the published formula.
- Cost figures are API list-price estimates reported by the tools; subscription plans are not billed per token.
- The author operates tokensave.app, a free token-cost calculator; the datasets are released to be checked and reused, and corrections are welcome.

6. Data availability

All datasets are released under CC BY 4.0 with CSV/JSON files, and most with raw logs and scripts. Concept DOIs resolve to the latest version.

#	Dataset	DOI	Page (tokensave.app/data/…)
1	Token cost by language: 41 languages measured on GPT tokenizers	10.5281/zenodo.23427148	token-cost-by-language
2	Token cost by language on Claude, Gemini and GPT	10.5281/zenodo.23428350	token-cost-by-language-claude-gemini
3	Token cost by data format	10.5281/zenodo.23435788	token-cost-by-format
4	Image input token cost by resolution	10.5281/zenodo.23443906	image-token-cost
5	Token cost by programming language	10.5281/zenodo.23446563	token-cost-by-programming-language
6	PDF token cost per page	10.5281/zenodo.23446565	pdf-token-cost
7	Claude effort levels	10.5281/zenodo.23446567	claude-effort-cost
8	Claude Code vs Codex: startup tokens, MCP tools and the same task	10.5281/zenodo.23448342	claude-code-vs-codex-tokens
9	Web search in Claude Code and Codex	10.5281/zenodo.23449842	ai-agent-web-search-tokens
10	AI web search by language	10.5281/zenodo.23451506	claude-web-search-by-language
11	CLAUDE.md and AGENTS.md size	10.5281/zenodo.23453217	claude-md-agents-md-size
12	Coding in your own language	10.5281/zenodo.23454308	ai-coding-prompt-language
13	What the Claude Code harness costs	10.5281/zenodo.23465633	claude-code-harness-cost

How to cite this report: TokenSave (2026). Where AI tokens go: 13 open measurements of LLM and coding-agent token costs. Technical report, version 1. Zenodo. https://doi.org/10.5281/zenodo.23467575. Contact: contact@jonhisking.com